

Práctica 10: Creación de librería Python personalizada e integración en suite

Metadatos

Campo	Detalle
Duración	72 minutos
Complejidad	Media
Nivel Bloom	Crear (Create)
Capítulo	5 — RF Avanzado, Data-Driven y Extensión con Python
Versión RF	Robot Framework 7.x

Descripción general

Cuando las keywords integradas (BuiltIn, Collections, OperatingSystem) no alcanzan para tu dominio específico, Robot Framework te permite escribir tus propias keywords en **Python puro**. En esta práctica vas a crear una librería con dos keywords para telecom (validar un número telefónico guatemalteco y calcular el costo total de un plan con impuesto), probarla con `pytest` **antes** de tocar Robot Framework, y luego integrarla en una suite.



Objetivos de aprendizaje

- Escribir funciones Python que se conviertan automáticamente en keywords.
 - Usar el decorador `@keyword` para personalizar nombre y tags.
 - Documentar una keyword con un docstring (Robot Framework lo usa como documentación oficial).
 - Probar la lógica con `pytest` antes de integrarla a una suite `.robot`.
-

Prerrequisitos

Área	Nivel
Python básico (funciones, import, re)	Básico
pytest (Sesión 1, Práctica 2)	Requerido

¿Cómo se convierte una función Python en keyword?

Cualquier función pública (sin `_` al inicio) de un archivo `.py` importado con `Library` se convierte automáticamente en una keyword. El nombre se traduce de `snake_case` a `Title Case Con Espacios`. Si quieres un nombre distinto, tags, o tipos de argumento, usas el decorador `@keyword` de `robot.api.deco`.

Pasos de la práctica

Paso 1 — Escribir la librería Python

Crea `libs/ValidadorTelecom.py`:

```
"""Librería Python personalizada para Robot Framework."""
from __future__ import annotations

import re

from robot.api.deco import keyword

NUMERO_TELEFONO_GT_PATTERN = re.compile(r"^[2-7]\d{7}$")

@keyword("Validar Numero De Telefono Guatemalteco")
def validar_numero_telefono(numero: str) -> bool:
    """Valida que un número guatemalteco tenga 8 dígitos y empiece
    entre 2 y 7.

    Example:
    | ${es_valido}=    Validar Numero De Telefono Guatemalteco    50212345
    | Should Be True    ${es_valido}
    """
    return bool(NUMERO_TELEFONO_GT_PATTERN.fullmatch(numero))

@keyword("Calcular Costo Total Del Plan", tags=["facturacion"])
def calcular_costo_total(costo_base: float, impuesto_porcentaje: float = 12.0) -> float:
```

```

"""Calcula el costo total de un plan agregando el impuesto (IVA de
Guatemala = 12% por defecto)."""
if costo_base < 0:
    raise ValueError(f"El costo base no puede ser negativo: {costo_base}")
total = costo_base * (1 + impuesto_porcentaje / 100)
return round(total, 2)

```

¿Qué hace @keyword("...", tags=[...])? Cambia el nombre visible de la keyword (sin eso, validar_numero_telefono se vería como Validar Numero Telefono, sin "De") y le asigna tags que aplicarán a cualquier test que la use directamente con [Tags] automáticos.

¿Por qué el docstring importa? Robot Framework lo usa como la documentación oficial de la keyword — visible en log.html y en el autocompletado del IDE.

Paso 2 — Probar la lógica con pytest (antes de tocar Robot Framework)

Crea tests_unitarios/test_validador_telecom.py:

```

from pathlib import Path
import sys
import pytest

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "libs"))
from ValidadorTelecom import calcular_costo_total, validar_numero_telefono

@pytest.mark.parametrize(
    "numero, esperado",
    [
        ("50212345", True),
        ("80123456", False), # fuera de rango
        ("1234567", False), # solo 7 dígitos
        ("abcdefgh", False), # no numérico
    ],
)
def test_validar_numero_telefono(numero, esperado):
    assert validar_numero_telefono(numero) is esperado

def test_calcular_costo_total_con_impuesto_por_defecto():
    assert calcular_costo_total(100) == 112.0

def test_calcular_costo_total_con_costo_negativo_lanza_value_error():
    with pytest.raises(ValueError):
        calcular_costo_total(-50)

pytest tests_unitarios/ -v

```

Salida esperada: todos los tests en verde.

💡 **¿Por qué probar con pytest antes de usarla en Robot Framework?** Porque así separas dos preguntas distintas: “¿mi lógica Python es correcta?” (pytest, rápido, sin depender de Robot Framework) y “¿la integración con Robot Framework funciona?” (la suite .robot del Paso 3). Si algo falla, sabes inmediatamente en cuál de las dos capas está el problema.

Paso 3 — Integrar la librería en una suite

Crea tests/validador_telecom_suite.robot:

```
*** Settings ***
Documentation      Integra la librería Python ValidadorTelecom como keywords.
Library            ../libs/ValidadorTelecom.py

*** Test Cases ***
Validar número telefónico guatemalteco correcto
    ${es_valido}=    Validar Numero De Telefono Guatemalteco    50212345
    Should Be True    ${es_valido}

Rechazar número telefónico con rango inválido
    ${es_valido}=    Validar Numero De Telefono Guatemalteco    80123456
    Should Not Be True    ${es_valido}

Calcular costo total de un plan con IVA por defecto
    ${total}=    Calcular Costo Total Del Plan    100
    Should Be Equal As Numbers    ${total}    112.0

Calcular costo total de un plan con impuesto personalizado
    [Tags]    facturacion
    ${total}=    Calcular Costo Total Del Plan    100    impuesto_porcentaje=10
    Should Be Equal As Numbers    ${total}    110.0

robot --outputdir reports tests/validador_telecom_suite.robot
```

Salida esperada: 4 tests, 4 passed, 0 failed.

Validación y pruebas

```
pytest tests_unitarios/ -v
robot --outputdir reports tests/validador_telecom_suite.robot
```

Lista de verificación final

Criterio	Estado
Los tests de pytest pasan en verde	☐
La suite .robot ejecuta 4 tests, 4 passed, 0 failed	☐
Should Be Equal As Numbers valida los cálculos con IVA	☐

Solución de problemas

ImportError: No module named 'ValidadorTelecom'

Causa: la ruta en Library ../libs/ValidadorTelecom.py no es correcta relativa a donde ejecutas robot. **Solución:** confirma que ejecutas el comando desde la carpeta de la práctica, y que la ruta relativa apunta al archivo correcto.

La keyword aparece como Validar Numero Telefono (sin “De”)

Causa: falta el decorador @keyword("Validar Numero De Telefono Guatemalteco"), o tiene un error de tipeo. **Solución:** revisa que el decorador esté justo arriba de la definición de la función.

Resumen

- Funciones Python públicas en un .py se convierten automáticamente en keywords.
- @keyword("Nombre", tags=[...]) personaliza el nombre visible y agrega tags.
- El docstring de la función es la documentación oficial de la keyword en Robot Framework.
- Prueba la lógica Python con pytest antes de integrarla — separa el error de lógica del error de integración.

Próximos pasos

En la **Sesión 6** vas a usar una librería de la comunidad (SeleniumLibrary) para automatizar flujos web completos.

Recursos

Recurso	URL
Creating test libraries (User Guide)	https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#creating-test-libraries
Decorador @keyword	https://robot-framework.readthedocs.io/en/stable/autodoc/robot.api.html#robot.api.deco.keyword